Software Engineering Assignment

-SDLC Model Selection

Software Engineering
Course Code- CSE1816OE11

Name: KUKIL NATH

Roll No: 230710007028

Semester: 6th Sem

Branch: COMPUTER SCIENCE & ENGINEERING

2026

1. Introduction

Project Selected: Payroll Management System

A Payroll System is a mission-critical enterprise software application responsible for automating and managing the financial records of employees' salaries, wages, bonuses, net pay, and deductions. The system must process inputs such as daily attendance records, leave balances, and overtime hours, and pass them through complex, legally bound business logic to calculate accurate payouts.
Key deliverables of this system include the generation of encrypted employee payslips, direct bank transfer integrations, and the automated calculation of statutory compliance deductions (such as Provident Fund, Income Tax, and health insurance premiums). Because it deals directly with company finances and employee livelihoods, the system demands absolute precision, stringent security, and zero tolerance for calculation errors in the production environment.

2. Selected SDLC Model

The Waterfall Model (Linear Sequential Life Cycle)
For the development of a fully compliant and accurate Payroll System, the Waterfall Model is the most appropriate Software Development Life Cycle methodology. The Waterfall model dictates that each phase of the development lifecycle (Requirements, Design, Implementation, Testing, Deployment, and Maintenance) must be fully completed and signed off before the next phase begins. There is no overlapping of phases, and the process flows strictly downwards, ensuring maximum stability and comprehensive documentation at every step.

3. Justification

The selection of the Waterfall Model for a Payroll System is driven by the rigid, mathematical, and legally bound nature of the software. Here are the five primary justifications:

- **1. Absolute Requirement Stability (Zero Ambiguity):** Payroll logic is not subjective; it is dictated by fixed mathematics, corporate HR policies, and government tax laws. Tax brackets, overtime multipliers, and statutory deduction percentages are known explicitly before a single line of code is written. Unlike a social media app where user trends dictate new features, the requirements for a payroll system do not change mid-development. Waterfall thrives when requirements are crystal clear, completely documented, and

frozen upfront.

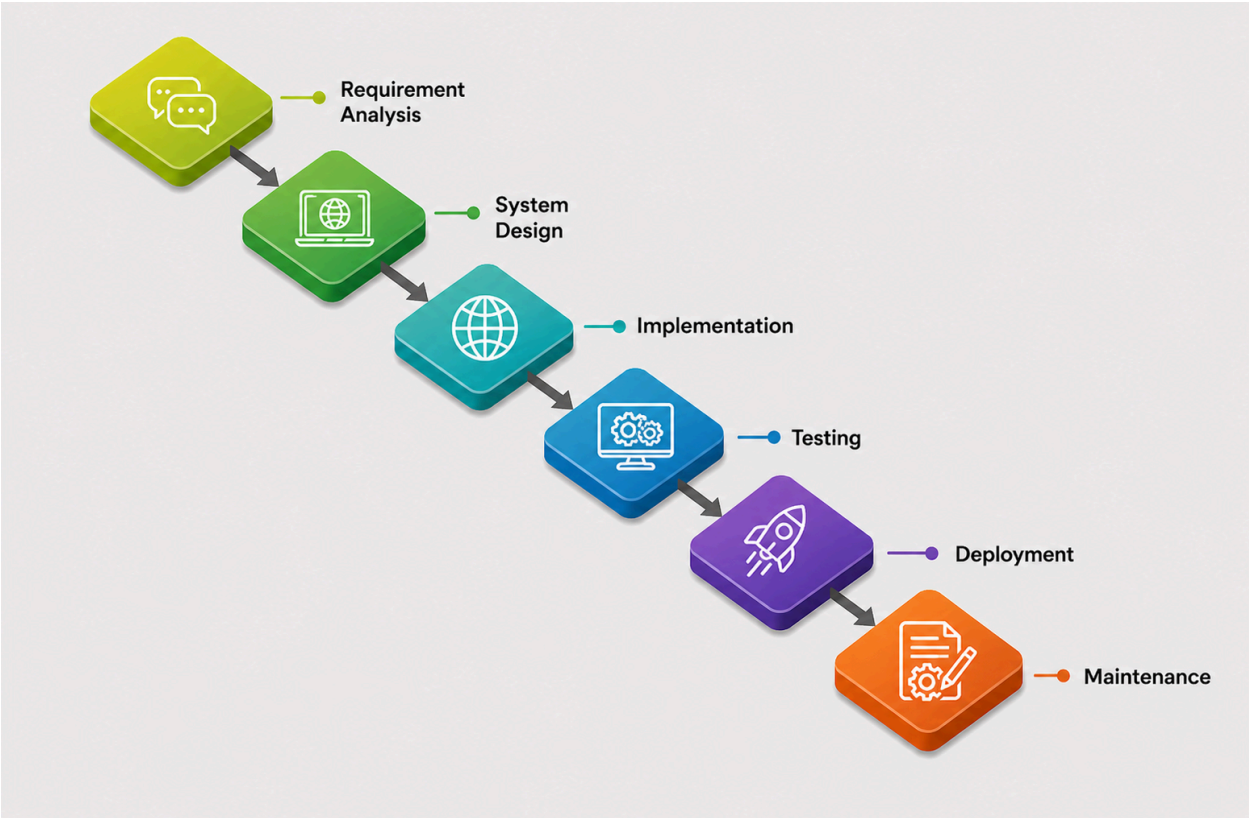
- **2. High Risk Level and Zero Tolerance for Production Bugs:** In financial software, releasing an incomplete or buggy product is catastrophic. An error in a payroll system could result in underpaying employees (causing labor disputes) or overpaying them (causing financial loss to the company), alongside severe legal penalties for incorrect tax filings. Waterfall's extensive, dedicated Testing phase—which occurs only after the entire system is designed and built—ensures that the complex calculation algorithms are rigorously verified against massive datasets before the system ever touches real money.
- **3. Algorithmic Complexity over UI/UX Fluidity:** A payroll system is computationally intensive but visually straightforward. The complexity lies entirely in the backend database architecture, security encryption, and calculation algorithms. It does not require a highly experimental or fluid User Interface (UI). Waterfall is highly suited for backend-heavy projects where the system architecture (database schemas, data flow diagrams) must be perfectly planned and finalized during the Design phase before implementation begins.
- **4. Minimal End-User Involvement During Development:** Once HR and Finance stakeholders define the payroll rules and formulas during the Requirement Gathering phase, there is no need for continuous user feedback. Employees do not need to "test drive" a payroll system iteratively; they simply need their final payslip to be 100% accurate. Waterfall is ideal here because it does not require the continuous user-involvement loops that are mandatory in Agile methodologies.
- **5. Inflexibility is a Feature, Not a Bug:** In many projects, Waterfall's rigid resistance to change is seen as a negative. For a Payroll System, it is a strict requirement. You cannot allow "scope creep" or shifting requirements when dealing with financial compliance. By utilizing Waterfall, any requested changes to the payroll logic after the requirements are locked require formal review, ensuring that a stray code change doesn't inadvertently break the entire tax calculation module.

4. Comparison with Other Models

- **Why Agile is Not Suitable:** Agile development relies on building a "Minimum Viable Product" (MVP) and releasing it in iterative sprints to gather user feedback. This approach is fundamentally incompatible with payroll. You cannot deploy an MVP payroll system that only calculates base salary but ignores tax deductions to "see how users like it." Releasing an incomplete financial system is illegal and operationally disastrous. Agile encourages changing requirements, which is exactly what a payroll system must avoid.
- **Why the Spiral Model is Not Suitable:** The Spiral model is highly focused on risk analysis

through the continuous building of prototypes (often used for unprecedented, highly experimental systems like aerospace software or novel AI). A payroll system is a well-understood, heavily documented domain. The mathematical rules of payroll do not require experimental prototyping to prove they work. Using the Spiral model would introduce unnecessary costs and delays for a system whose mechanics are already a known quantity.

5. Diagram & Step-by-Step Breakdown



The Waterfall Phase Breakdown for Payroll Systems:

1. **Phase 1: Requirements Analysis** - Business analysts meet with the Finance and HR departments. Every single tax rule, leave deduction formula, and overtime policy is documented in a Software Requirements Specification (SRS) document. The requirements are then frozen.
2. **Phase 2: System Design** - Database architects design the employee tables, attendance relational schemas, and security protocols (e.g., encryption for bank details). The architecture is mapped out completely before coding starts.

3. **Phase 3: Implementation** - Software engineers write the code based strictly on the design documents. The calculation engines, API integrations for banking, and PDF generators for payslips are programmed.
4. **Phase 4: Testing** - Quality Assurance (QA) engineers run extensive test cases, passing dummy employee data through the system to ensure calculations match manual ledger math perfectly. Edge cases (e.g., an employee taking unpaid leave on a public holiday) are verified.
5. **Phase 5: Deployment** - Once certified bug-free, the system is installed on the company's secure servers and integrated with the live HR database.
6. **Phase 6: Maintenance** - The system runs in production. Updates are only made when government tax brackets change annually, requiring patches to the calculation logic.

6. Conclusion

In conclusion, the **Waterfall model** is the ideal SDLC choice for developing this **Payroll Management System**. Given that the project's requirements are rigidly defined by immutable mathematical algorithms, complex tax regulations, and fixed corporate policies, there is zero ambiguity that needs iterative clarification. The linear nature of Waterfall provides the necessary structure to enforce comprehensive planning, thorough system design, and rigorous testing against massive datasets. It ensures computational precision is verified *before* deployment, minimizing the risk of catastrophic financial or legal errors. While less flexible for dynamic environments, for a project prioritizing structural stability, absolute compliance, and zero failure in production like payroll, Waterfall offers the safest and most efficient path to success.